

Guía Didáctica: Promesas en JavaScript

Apartado 4: Promesas Encadenadas (Chaining) y uso avanzado

Explicación

El **encadenamiento de promesas** (chaining) permite ejecutar varias operaciones asíncronas de manera secuencial, donde el **resultado de una promesa se pasa como entrada a la siguiente**.

- Cada `then()` recibe el valor devuelto por el `then()` anterior.
- Si alguna promesa falla, el control pasa directamente al `catch()`.
- Permite **evitar el “callback hell”**, haciendo el código más legible y mantenible.

Sintaxis básica:

```
promesa()
  .then(resultado1 => {
    return resultado1 + 1;
  })
  .then(resultado2 => {
    return resultado2 * 2;
  })
  .then(resultadoFinal => {
    console.log("Resultado final:", resultadoFinal);
  })
  .catch(error => console.error(error));
```

Ejemplo práctico 1: Encadenamiento simple

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Promesas Encadenadas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

<h1>Ejemplo 1: Promesas Encadenadas</h1>
<button id="btnChain1" class="btn btn-primary">Ejecutar
Chaining</button>
<div id="resultadoChain1" class="mt-3"></div>

<script>
const btnChain1 = document.getElementById('btnChain1');
const resultadoChain1 = document.getElementById('resultadoChain1');
```

```

btnChain1.addEventListener('click', () => {
  new Promise((resolve, reject) => {
    setTimeout(() => resolve(5), 1000);
  })
  .then(num => num * 2)
  .then(num => num + 3)
  .then(resultado => {
    resultadoChain1.innerHTML = `<div class="alert alert-
success">Resultado final: ${resultado}</div>`;
  })
  .catch(err => {
    resultadoChain1.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
  });
});
</script>

</body>
</html>

```

Explicación:

- Se encadenan tres operaciones asíncronas: multiplicar → sumar → mostrar resultado.
- El flujo es secuencial y fácil de leer.

Ejemplo práctico 2: Encadenamiento con fetch API

```

<h2>Ejemplo 2: Chaining con Fetch</h2>
<button id="btnChain2" class="btn btn-success">Cargar Usuarios y sus
Posts</button>
<div id="resultadoChain2" class="mt-3"></div>

<script>
const btnChain2 = document.getElementById('btnChain2');
const resultadoChain2 = document.getElementById('resultadoChain2');

btnChain2.addEventListener('click', () => {
  fetch('https://jsonplaceholder.typicode.com/users/1')
    .then(response => response.json())
    .then(user => {
      resultadoChain2.innerHTML = `<div class="alert alert-
info">Usuario: ${user.name}</div>`;
      return
      fetch(`https://jsonplaceholder.typicode.com/posts?userId=${user.id}`);
    })
    .then(response => response.json())
    .then(posts => {
      let html = '<ul class="list-group">';
      posts.forEach(post => html += `<li class="list-group-
item">${post.title}</li>`);
      html += '</ul>';
      resultadoChain2.innerHTML += html;
    })
    .catch(err => {

```

```

        resultadoChain2.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
    });
});
</script>

```

Explicación:

- Primer fetch obtiene un usuario.
- Segundo fetch depende del ID del usuario obtenido previamente.
- Encadenar promesas evita anidamiento profundo de callbacks.

Ejemplo práctico 3: Encadenamiento con manejo de errores

```

<h2>Ejemplo 3: Chaining con Catch</h2>
<button id="btnChain3" class="btn btn-warning">Ejecutar
Promesas</button>
<div id="resultadoChain3" class="mt-3"></div>

<script>
const btnChain3 = document.getElementById('btnChain3');
const resultadoChain3 = document.getElementById('resultadoChain3');

btnChain3.addEventListener('click', () => {
    new Promise((resolve, reject) => {
        const exito = Math.random() > 0.5;
        setTimeout(() => exito ? resolve(10) : reject("Error
inicial"), 1000);
    })
    .then(num => {
        return num * 3;
    })
    .then(num => {
        if(num > 20) throw "Número demasiado grande!";
        return num + 5;
    })
    .then(resultado => {
        resultadoChain3.innerHTML = `<div class="alert alert-
success">Resultado final: ${resultado}</div>`;
    })
    .catch(err => {
        resultadoChain3.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
    });
});
</script>

```

Explicación:

- Captura errores tanto en la promesa inicial como en operaciones posteriores.
- Evita romper la cadena y permite manejar errores en un solo `catch()`.

Ejemplo práctico 4: Chaining con fetch POST y GET

```
<h2>Ejemplo 4: Chaining POST y GET</h2>
<button id="btnChain4" class="btn btn-info">Enviar y
Consultar</button>
<div id="resultadoChain4" class="mt-3"></div>

<script>
const btnChain4 = document.getElementById('btnChain4');
const resultadoChain4 = document.getElementById('resultadoChain4');

btnChain4.addEventListener('click', () => {
  fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: 'Nuevo Post', body: 'Contenido
de prueba', userId: 1 })
  })
  .then(response => response.json())
  .then(data => {
    resultadoChain4.innerHTML = `<div class="alert alert-
success">Post creado con ID: ${data.id}</div>`;
    return
  fetch(`https://jsonplaceholder.typicode.com/posts/${data.id}`);
  })
  .then(response => response.json())
  .then(post => {
    resultadoChain4.innerHTML += `<div class="alert alert-
info">Post obtenido: ${post.title}</div>`;
  })
  .catch(err => {
    resultadoChain4.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
  });
});
</script>
```

Explicación:

- Primer fetch POST crea un recurso en el servidor.
- Segundo fetch GET obtiene el recurso recién creado.
- Todo manejado de forma secuencial mediante chaining.

Ejemplo práctico 5: Chaining con múltiples fetch simultáneos (Promise.all)

```
<h2>Ejemplo 5: Chaining con Promise.all</h2>
<button id="btnChain5" class="btn btn-secondary">Cargar Datos
Simultáneos</button>
<div id="resultadoChain5" class="mt-3"></div>

<script>
const btnChain5 = document.getElementById('btnChain5');
const resultadoChain5 = document.getElementById('resultadoChain5');

btnChain5.addEventListener('click', () => {
```

```

const urls = [
  'https://jsonplaceholder.typicode.com/users/1',
  'https://jsonplaceholder.typicode.com/users/2'
];

Promise.all(urls.map(url => fetch(url).then(res => res.json()))).then(results => {
  let html = '';
  results.forEach(user => {
    html += `<div class="alert alert-info">Usuario:
    ${user.name}, Email: ${user.email}</div>`;
  });
  resultadoChain5.innerHTML = html;
}).catch(err => resultadoChain5.innerHTML = `<div class="alert alert-danger">${err}</div>`);
});
</script>

```

Explicación:

- `Promise.all()` permite ejecutar múltiples promesas simultáneamente y esperar a que todas se completen.
- Útil para **peticiones paralelas al servidor**.

Ejercicios prácticos resueltos para alumnos

1. Crear una cadena de promesas que tome un número, lo multiplique y le sume un valor, mostrando el resultado final.
2. Encadenar un fetch GET de usuario y un fetch GET de sus posts, mostrando los datos en pantalla.
3. Crear una cadena de promesas con posibles errores y capturarlos con `catch()`.
4. Usar `Promise.all()` para obtener datos de varios usuarios simultáneamente y mostrarlos en una lista.

Ejercicios propuestos para alumnos

1. Encadenar promesas que simulen pasos de un proceso (ej. cargar datos → procesar datos → mostrar resultado).
2. Usar fetch POST para crear un recurso y luego fetch GET para mostrarlo, usando chaining.
3. Crear un proceso encadenado que valide datos de un formulario y luego los envíe al servidor.
4. Usar `Promise.all()` para cargar datos de tres APIs públicas al mismo tiempo y mostrarlos en pantalla.